Hes·so VALAIS WALLIS
Haute Ecole d'Ingénierie
Hochschule für Ingenieurwissenschaften

Nom : _____

Prénom : _____

Classe : _____

SOLUTIONS EN ROUGE

Examen intermédiaire 2 – Embedded systems 1 - Embedded programming

Question / points	1 / 3	2 / 3	3 / 5	4 / 12	5 / 12	Total / 35
Résultat						

Consigne :

Lisez attentivement la donnée et répondez de manière **lisible** aux questions. Le cours et la documentation de laboratoire sont autorisés.

Question 1 (le PIC): (3 points)

Répondez aux questions.

Sur le PIC18F97J60, il y a 1 convertisseurs AD et 16 entrées analogiques?

Un convertisseur AD 12 bits retourne des valeurs entre 0 et 4095

Sur un UART à 9'600 bauds il faut au minimum 1041 microsecondes pour transmettre un byte.

La norme UART implique une transmission série, MSb first ? FAUX

Le bit de parité sert à resynchroniser la vitesse de transmission ? FAUX

Les interruptions de haute priorité du PIC18 peuvent réveiller le MCU en mode Sleep ? VRAI

Question 2 (le C): (3 points)

Répondez aux questions.

Une variable de type « union » de 3 champs `uint16_t` occupera 2 bytes en mémoire.

Un pointeur n'est pas une variable ? FAUX

Un pointeur est toujours initialisé à « `Null` » ? FAUX

Le référencement « & » permet d'accéder au contenu d'un pointeur ? FAUX

Quelle place occupera la variable **mySystem** ci-dessous ? 15 à 17 bytes (dépend de ptr)

Quelle place occupera la variable **myPtr** ci-dessous ? 2 à 4 bytes

```
typedef struct {
  uint8_t mode ;
  char sys[12] ;
  uint8_t * pIn ;
}System ;
System mySystem ;
System * myPtr = &mySystem ;
```

Question 3: (5 points)

Trouvez les 4 erreurs dans le code proposé ci-dessous et corrigez ce qui ne va pas.

```
typedef struct {
  uint8_t count ;
  uint8_t sys[12] ;
  uint8_t * pIn ;
}System ;

//---------------------------------------------------------------------------
void systemInit(System * system)
{
  system.count = 0;                // set count to 0
  for(uint8_t i=0;i<=12;i++)       // c'est 11 et pas 12
  {
    system.sys[i] = 0;             // set sys table elements to 0
  }
  system.pIn = &system.sys[0];      // point to first element of sys
}

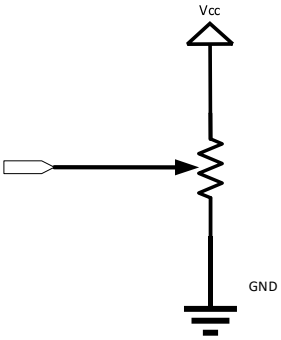
//---------------------------------------------------------------------------
void systemSetEntry(System * pSys,uint8_t val)
{
  *pSys->pIn = val;                 // set value in pointed element
  pSys->count++;                   // incement counter
  if(pSys->count < 12)             // if end of sys table not reached
  {
    pSys->pIn++;                   // increment pointer to next element
  }
  else
  {
    pSys->pIn = &pSys->sys;        // else, reset pointer to first element
    pSys->count = 0;               // reset count to 0
  }
}

//---------------------------------------------------------------------------
void main(void)
{
  System mySystem ;

  systemInit(&mySystem) ;             // initialize mySystem
  systemSetEntry(&mySystem, 4) ;     // insert an element
  ...
}
```

Question 4: (12 points)

Soit le potentiomètre suivant (voir figure ci-dessous) mécaniquement relié à la pédale d'accélérateur d'une maquette de simulation de voiture. La conception mécanique ne permet pas à la pédale de faire varier le potentiomètre sur la totalité de sa plage de valeurs.



Nous désirons pouvoir mesurer la position de l'accélérateur, à tout moment, de sa position minimum jusqu'à sa position maximum et retourner une valeur entre 0 et 100%.
Etant donné la structure "`Pedal`" existante, écrivez la fonction "`PedalMeasure`()" qui retourne une valeur d'accélération entre 0 et 100. Attention, les valeurs min_ADC et max_ADC doivent être mises à jour si nécessaire dans cette fonction. Utilisez la fonction `ADC_Measure` qui vous retournera la valeur mesurée de l'ADC 10 bits.

```
uint16_t ADC_Measure(uint8_t channel);

typedef struct {
    uint16_t minADC;            // min value measured on pedal
    uint16_t maxADC;            // max value measured on pedal
    uint16_t adResult;          // position of pedal (0-100)
}Pedal;
```

L'appel de cette fonction depuis le « main » se fera de la manière suivante :

```
percentPedal = PedalMeasure(&myPedal,0);
```

Au départ, la fonction main aura au préalable initialisé la variable myPedal avec les valeurs ci-dessous :
myPedal.minADC = 65535 ;
myPedal.maxADC = 0 ;

```
...
typedef struct {
    uint16_t minADC;            // min value measured on pedal
    uint16_t maxADC;            // max value measured on pedal
    uint16_t adResult;          // last ADC result
} Pedal;
Pedal myPedal;

uint8_t PedalMeasure(Pedal * pedal, uint8_t channel)
{

   uint8_t precent;

   adResult = ADC_Measure();
   if(adResult < myPedal->minADC)
   {
      myPedal->minADC = adResult;
   }
   else if(adResult > myPedal->maxADC) // else is important !!!
   {
      myPedal->maxADC = adResult;
   }
   }
   percent = (((uint32_t)adResult - pedal->minADC)*100) /
             (pedal->maxADC - pedal->minADC);









}
```

Question 5: (12 points)

Soit une communication Modbus RTU que vous ne connaissez pas mais dont le protocole est expliqué ci-dessous.
La trame Modbus qui nous intéresse est représentée ci-dessous. Elle est composée de 8 bytes.

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 7	Byte 7
Address	Function	**RegH**	**RegL**	**ValH**	**ValL**	CRCH	CRCL

Ces bytes seront reçus en interruption par l'UART du PIC qui sera déjà configuré en conséquences. La variable modbusRec de type union décrite ci-dessous sera utilisée.
Dans votre propre Interrupt Handler (`MyReceiveHandler`), tout d'abord, le tableau de bytes sera rempli à chaque caractère reçu. En plus, si un message complet a été reçu (les 8 bytes reçus), le tableau singleReg sera mis à jour en fonction de la valeur **Val** de la trame dans son numéro de registre **Reg**.

```
typedef struct {
    uint8_t address;            // address of Modbus server
    uint8_t function;           // function code
    uint16_t register;          // register address
    uint16_t value;             // register value
    uint16_t CRC;               // CRC control
}Frame;

typedef union {
    uint8_t recBuf[8];          // frame bytes
    Frame frame;                // logic access
}ModbusRec;

ModbusRec modbusRec;            // frame variable
uint16_t singleRec[10];         // Modbus registers
```

```
void MyReceiveHandler(void)
{
  static uint8_t recCount = 0;        // counter of received bytes
  uint8_t recByte = RCREG1;           // last received byte

  modbusRec.recBuffer[recCount] = recByte;
  recByte++ ;
  if(recByte == 8)
  {
    // swap because little & big endian
    modbusRec.frame.register = (modbusRec.frame.register >> 8)|
                               (modbusRec.frame.register << 8);
    // idem for value
    modbusRec.frame.value = (modbusRec.frame.value >> 8)|
                            (modbusRec.frame.value << 8);
    singleRec[modbusRec.frame.register] =
                              modbusRec.frame.value;
    recByte = 0;
  }









}
```